

Closet Quest Technical Feasibility and Baseline

Sprint 3 submission | Junhao Yan | Repository owner noobyanjunhao
Repository: <https://github.com/noobyanjunhao/Closet-Quest>

Decision: Modify. Continue the closet, styling, quest, and resale workflow, but replace independent garment selection with constrained outfit ranking. Two experiments establish local technical feasibility: a reproducible metadata benchmark and a CPU-only pretrained-vision smoke test. They do not establish human styling acceptance or real-photo recognition accuracy. Keep manual corrections and defer a final model or fine-tuning decision until a real-photo evaluation is reviewed with Junhao.

Technical and data feasibility

The highest product risk is that digitization creates more correction work than value. A second critical assumption is that small, incomplete closets still support useful context-specific outfits. Sprint 3 directly tests the latter with representative metadata scenarios and tests whether existing vision models can execute locally without training. The semester targets remain category accuracy $\geq 80\%$, color accuracy $\geq 85\%$, valid-job success $\geq 95\%$, 90% of uploads within 20 seconds, and human acceptance within the top three $\geq 70\%$.

Risk and priority	Dependencies and access	Assessment and mitigation
High: photo recognition and cleanup	Consented labeled photos; pretrained model weights; inference runtime; separate segmentation tool.	Only illustration inference tested. No color or cleanup validation. Keep all attributes editable; compare models before tuning.
High: useful styling from sparse closets	Reliable categories/context tags, garment ownership and wear counts; human ratings.	Test shape, context, low-use constraints and abstention. Wrong metadata still causes missed outfits.
High: private persistent wardrobe	Authentication, PostgreSQL, private object storage, API authorization and image deletion.	Current browser storage is a demo, not account isolation. Prove cross-user access denial before hosted private data.
Medium: latency and reliability	CPU/RAM, downloads, bounded ranking, job timeouts/retries; production HTTPS.	Local measurements only. Separate image jobs from normal API calls and retain failure/retry states.
Medium: external dependencies	Node/npm lockfile, ONNX runtime; model licenses and network access.	No GPU or paid API used. Models cached outside Git. RMBG-2.0 is gated/noncommercial; not integrated. [1-3]
Medium: resale correctness	User-confirmed size/condition/price; stored image; export destination.	Editable export only. Do not invent condition, auto-post, or add a marketplace/payment system.

Infrastructure direction: React web client now; proposed FastAPI service later with PostgreSQL and object storage. Deploy with HTTPS, restart handling and memory planning; these are separate tasks from running a development server. [4] Expo remains an option after the workflow is validated. No production hosting, GPU capacity, or external account access is claimed.

Baseline and feasibility prototype

Experiment A Outfit feasibility with representative metadata

Hypothesis: a rule-based ranker can return complete, owned, context-matching outfits, include a low-use garment when required, and abstain when no solution exists, without a trained recommendation model. The original baseline is frozen from repository commit dab74ac. It chooses each category separately, favors low wear counts before context, and cycles all slots together. The prototype searches complete outfits, normalizes exact context tags, and exposes three distinct options with explanations.

Data: 30 original synthetic wardrobes: ten scenario families with three seeded wear-count variants each. Each wardrobe receives three contexts and ordinary/low-use requests, giving 180 queries. Scenarios cover balanced closets, context conflicts, dress-only wardrobes, missing shoes, sparse inventory, unknown tags, all-used clothing, mixed-case tags, dress alternatives, and mislabeled footwear. These are inspectable engineering scenarios, not observations from wardrobe owners. The variants and requests are correlated.

Reference rule: an independently implemented brute-force oracle accepts one top plus one bottom, or one dress, with exactly one pair of shoes and optional outerwear. All pieces must be owned and tagged for the requested context; quest requests additionally require a piece with ≤ 1 wear. A feasible request succeeds if any of up to three suggestions passes. Impossible requests are evaluated separately for correct abstention. This proxy is deliberately narrower than fashion quality.

Measurement: five warmups and 30 timed repeats per query (5,400 calls per method); nearest-rank P50/P95 from warm in-process CPU calls. Stress tests use 50, 100 and 200 garments, two warmups and ten measured calls per size. The small-sample stress P95 is the observed maximum. No API, storage, upload or model latency is included in these ranking timings.

Experiment B Pretrained vision execution without training

Compare quantized CLIP ViT-B/32 and SigLIP Base ONNX checkpoints using Transformers.js on CPU. [1,2] The fixture is 18 original garment illustrations: six shapes with three color/background variations, derived from the app SVG assets. Anonymous filenames prevent category leakage. Reference categories come from the authored shapes. The existing filename heuristic defaults all 18 images to Top. These images represent demo silhouettes, not real wardrobe photos.

Both models use the same six fixed label phrases and template, one warmup, then one timed prediction per image. Record every prediction, ranking score, latency and pinned model revision; no prompts, weights or thresholds are tuned. Use q8, four intra-op threads and one inter-op thread. Scores are uncalibrated rankings. No model performs background cleanup or color extraction in this test.

Reproduction and available resources

```
npm ci
npm test
npm run benchmark
npm run vision:compare
npm run dev
```

Measured host: AMD Ryzen 9 8945HS w/ Radeon 780M Graphics; 16 logical CPUs; 15 GiB reported RAM; Windows x64; Node v24.19.0. No discrete GPU required. npm and first-time model downloads need network access. The lockfile, dataset hashes, model revisions, source, raw results, and generators are checked in; model caches are ignored. Inspect the comparison in the app's Lab screen.

Measured findings and limitations

Outfit constraints improve while ownership is preserved

Metric and denominator	Original baseline	Ranked prototype
Feasible requests with valid top-three result	78/98 (79.6%)	98/98 (100.0%)
Impossible requests correctly declined	36/82	82/82
Constraint-invalid individual suggestions	304/432	0/263
Suggestions entirely from stored garments	432/432	263/263
Duplicate suggestions within a request	36	0
Warm ranking P95 across fixture calls	0.011 ms	0.036 ms

The baseline already preserves ownership, but frequently suggests garments that fail context or quest constraints. Its top-three success rate hides invalid individual options and impossible requests. The prototype removes those failures in this fixture. Unknown and mislabeled metadata produce explicit abstention; this is safe behavior under recorded metadata, not proof that the actual closet lacks a wearable combination.

A scalability failure changed the implementation

The first exhaustive prototype took 2,613.2 ms at P95 for 200 garments, exceeding a one-second interaction budget. The revised search keeps the best candidates per slot under the separable wear score; its 200-item P95 was 0.334 ms. Tests establish equality with exhaustive top-three outputs on all 180 fixture queries. This pruning depends on the current additive score and must be reconsidered if pairwise color/style compatibility is added. Both runs and the failed exhaustive implementation are preserved.

Vision smoke test results

Method	Correct illustrations	Warm P95
Filename heuristic	3/18 (16.7%)	Not timed
CLIP ViT-B/32 q8 CPU	18/18 (100.0%)	40.2 ms
SigLIP Base q8 CPU	17/18 (94.4%)	159.9 ms

SigLIP labeled one outerwear illustration as Top. CLIP is the provisional next-test candidate on this evidence because it made fewer errors and ran faster. Eighteen correlated, clean illustrations are too easy and too small to establish real-photo accuracy or statistical superiority. Each model is forced to select one of six classes even for an unrelated photo. Cached load time and download time are separate from warm inference. No job-failure, real upload, or mobile-device benchmark was performed.

Evidence: experiments/results/latest.json, experiments/results/exhaustive-prototype.json, and vision/results/latest.json include full outputs and timing protocol. The benchmark quality counts are deterministic; timings vary with device load. There is no held-out user or photo evaluation and no confidence interval presented for correlated synthetic samples.

Project decision and next validation

Modify the implementation and proceed with bounded scope

The evidence supports a working technical path for stored-garment styling and local pretrained classification. It does not justify claiming all semester Must Have requirements are complete. Adopt constrained ranking, visible abstention and metadata correction now. Keep the web demo and resale export. Delay social features, weather, marketplace integration, large-model training and mobile migration until the core workflow is measured with users.

Next step	Owner and acceptance gate
Ship this Sprint 3 baseline	Implemented: ranked options, explanations, stricter quest validation, Lab comparison, reproducible scripts and raw evidence. Maintain regression checks.
Evaluate real photos before choosing a model	Junhao and project team: collect ≥ 120 distinct, consented garments across six classes. Use 60 development and 60 locked test items, grouping alternate views by garment. Compare both pretrained models; review confusion matrix, macro F1, unknown-item failures, correction burden and $\geq 80\%$ category target.
Decide on tuning together	Junhao participates in model/prompt selection and any tuning. If zero-shot performance is inadequate, jointly consider prompt changes or a frozen-embedding classifier before fine-tuning. Do not tune on locked test labels.
Validate actual styling usefulness	Recruit 5-8 target users for an initial pilot. Randomize/blind baseline and prototype options and record accept/modify/reject, task completion and correction time. Evaluate the $\geq 70\%$ top-three acceptance target with explicit denominators; do not invent results.
Connect and validate backend	Add authentication, private storage and complete deletion. Test cross-user access denial, image-job retries, $\geq 95\%$ job success and 90% under 20 seconds. Measure deployed API P95 < 1 second separately.

Contingency: if photo recognition or background removal remains unreliable, retain manual metadata and original images as an explicitly scoped fallback while collecting correction data. This changes the original AI-processing commitment and must be agreed by the team/instructor; it is not equivalent to completing it. If real users reject constrained outfits, revise ranking or narrow contexts before adding personalization.

Submission and inspection

Upload this four-page PDF to the course assignment. The working proof of concept is in github.com/noobyanjunhao/Closet-Quest. README and experiments/README.md document setup, scenarios, metrics, limitations and the preserved performance failure. vision/README.md documents the pretrained comparison and collaborative evaluation plan. The report builder reads the checked-in evidence; no course submission is automated.

References and technical sources

- [1] Hugging Face Xenova. CLIP ViT-B/32 ONNX model card and usage. huggingface.co/Xenova/clip-vit-base-patch32
 - [2] Google and Hugging Face Xenova. SigLIP Base model and ONNX usage. huggingface.co/google/siglip-base-patch16-224; huggingface.co/Xenova/siglip-base-patch16-224
 - [3] BRIA. RMBG-2.0 model card and access conditions. huggingface.co/briaai/RMBG-2.0
 - [4] FastAPI. Deployment concepts. fastapi.tiangolo.com/deployment/concepts/
- Accessed September 13 2026. Project targets come from the supplied Semester MVP Report; measured results come from this repository.